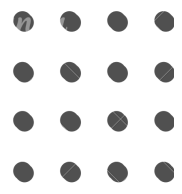




VMES Grinds

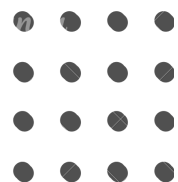




VMES Grinds

Pilot Program - Week 6 - Day 1

Graph • BFS • DFS





LeetCode: 997 Find the Town Judge



Moe Myint Mo San



997. Find the Town Judge

Solved

Easy



Topics



Companies

In a town, there are n people labeled from 1 to n . There is a rumor that one of these people is secretly the town judge.

If the town judge exists, then:

1. The town judge trusts nobody.
2. Everybody (except for the town judge) trusts the town judge.
3. There is exactly one person that satisfies properties 1 and 2.

You are given an array `trust` where `trust[i] = [ai, bi]` representing that the person labeled a_i trusts the person labeled b_i . If a trust relationship does not exist in `trust` array, then such a trust relationship does not exist.

Return the label of the town judge if the town judge exists and can be identified, or return -1 otherwise.



Example 1:

Input: `n = 2, trust = [[1,2]]`

Output: `2`

Example 2:

Input: `n = 3, trust = [[1,3],[2,3]]`

Output: `3`

Example 3:

Input: `n = 3, trust = [[1,3],[2,3],[3,1]]`

Output: `-1`



Constraints:

- $1 \leq n \leq 1000$
- $0 \leq \text{trust.length} \leq 10^4$
- $\text{trust}[i].\text{length} == 2$
- All the pairs of `trust` are **unique**.
- $a_i \neq b_i$
- $1 \leq a_i, b_i \leq n$



Problem Explanation Thought Process





Initial Thought Process

- This is not a traversal problem.
- We do not need BFS or DFS.
- We only care about:
 - Who trusts whom
- Translate conditions into counts:
 - Judge has 0 outgoing trust
 - Judge has $n - 1$ incoming trusts



Algorithm Evolution Overview





Approach	Time	Space	Notes
Brute Force (Graph Check)	$O(n^2)$	$O(n)$	Check each person manually
Degree Counting	$O(n + m)$	$O(n)$	Optimal and clean



Brute Force





```
class Solution(object):
    def findJudge(self, n, trust):
        if n == 1:
            return 1

        # build adjacency list (who each person trusts)
        graph = [[] for _ in range(n + 1)]
        for a, b in trust:
            graph[a].append(b)

        # check each person as potential judge
        for candidate in range(1, n + 1):
            # condition 1: judge trusts nobody
            if graph[candidate]:
                continue

            trusted_by_all = True
            for other in range(1, n + 1):
                if other == candidate:
                    continue
                if candidate not in graph[other]:
                    trusted_by_all = False
                    break

            if trusted_by_all:
                return candidate

        return -1
```



Time Complexity (Brute Force)

- Time: $O(n^2)$
- Space: $O(n)$



Degree Counting





Idea:

- Use an array `trust_count`
- For each trust `[a, b]`:
 - `a` trusts someone $\rightarrow$ decrement
 - `b` is trusted $\rightarrow$ increment
- The judge must end with:
 - `trust_count[i] == n - 1`

Why It Works

- Judge trusts nobody $\rightarrow$ no decrement
- Everyone else trusts judge $\rightarrow$ +1 from each person
- Final value:
 - `n - 1` confirms all conditions
- No graph traversal needed



```
class Solution(object):
    def findJudge(self, n, trust):
        if n == 1:
            return 1

        trust_count = [0] * (n + 1)

        for a, b in trust:
            trust_count[a] -= 1
            trust_count[b] += 1

        for i in range(1, n + 1):
            if trust_count[i] == n - 1:
                return i

        return -1
```




Time Complexity (Brute Force)

- Time: $O(n + m)$
 - m = number of trust relationships
- Space: $O(n)$
 - Trust count array



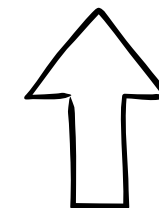
Complexity Comparison



Approach	Time	Space	Notes
Brute Force	$O(n^2)$	$O(n)$	Conceptually simple, inefficient
Degree Counting	$O(n + m)$	$O(n)$	Optimal and interview-safe



Extra Study Material –Visualizations–





Key Takeaway





This problem is about relationship counting, not traversal:

- Judge:
 - Trusts nobody
 - Trusted by everyone
- Translate rules directly into:
 - In-degree and out-degree counts
- Use counting when search is unnecessary



Thank You !

